

4장 메시지는 왜 사라지기도 하고 두 번 오기도 하는가

4주차 · Kafka

3장에서 브로커(broker, 이벤트를 저장하고 내주는 서버)를 띄웠다면, 이 장은 그 위로 민원 이벤트 한 건이 지나가는 경로를 따라간다. 접수 순서가 어디까지 지켜지는지, 그리고 같은 시스템에서 왜 어떤 메시지는 조용히 사라지고 어떤 메시지는 두 번 오는지를 실행 결과로 확인한다.

이 장의 수치는 로컬 브로커에 민원 이벤트를 실제로 보내고 받은 기록(`ch4_delivery_report.json`)에서 그대로 옮긴 것이다.

1. 오늘의 질문

- 민원이 접수된 순서는 시스템 안에서 어디까지 지켜질까?
- 같은 키(key)로 묶으면 부하도 고르게 나뉘까?
- 왜 메시지가 오류 한 줄 없이 사라지고, 왜 같은 메시지가 두 번 올까?
- 유실과 중복 중 하나를 반드시 골라야 한다면 공공 데이터는 어느 쪽을 택할까?

2. 민원실 창구로 보는 이 장

이 장의 개념은 창구가 여러 개인 민원실에 하나씩 대응한다.

민원실	Kafka
접수 업무 한 종류를 처리하는 민원실	토픽(topic) — 같은 종류의 이벤트를 담는 이름 붙은 단위
민원실 안의 창구 하나와 그 앞의 줄	파티션(partition) — 순서가 매겨진 로그 하나
어느 창구로 보낼지 정하는 배정 규칙	키와 파티셔닝(partitioning) — 같은 키는 늘 같은 파티션
창구별로 따로 뽑는 번호표 번호	오프셋(offset) — 파티션 안의 순번
창구 기록부의 "여기까지 처리함" 줄	커밋된 오프셋(committed offset)
담당자가 교대해도 기록부부터 이어 받는 것	재시작한 소비자가 커밋 위치부터 다시 읽는 것

여기서 두 가지가 따라 나온다. 번호표가 창구마다 따로 매겨지듯 순서는 창구 하나 안에서만 정의되고, 담당자가 기록부에 "처리함"을 언제 적느냐에 따라 교대 시점의 사고가 갈린다. 처리 전에 적어 두고 쓰러지면 하지 않은 일이 한 것으로 남고, 처리하고 적기 전에 쓰러지면 다음 담당자가 같은 일을 다시 한다.

비유가 깨지는 지점은 하나다. 창구에서 처리한 민원인은 줄에서 사라지지만, Kafka의 로그는 읽어도 지워지지 않는다. 읽는 위치를 되돌려 과거 이벤트를 다시 처리하는 리플레이(replay)가 가능한 것이 이 차이에서 나온다.

3. 순서는 파티션 안에서만 지켜진다

민원을 받는 쪽과 처리하는 쪽의 속도는 거의 항상 다르다. 재난 문자를 보낸 직후 접수 API는 초당 수백건을 받는데 처리 서버는 그 속도를 따라가지 못한다. 두 쪽을 직접 연결하면 이 속도 차이가 곧 요청 거부와 데이터 유실이 된다.

Kafka는 둘 사이에 추가 전용 로그(append-only log)를 둔다. 생산자(Producer)는 이벤트를 로그 끝에 붙이기만 하고, 소비자(Consumer)는 자기 속도로 읽는다. 이 단순한 구조가 버스트를 흡수하는 버퍼, 과거로 되돌아가는 리플레이, 그리고 생산자가 소비자의 수와 생사를 몰라도 되는 분리(decoupling)를 만든다.

토픽은 여러 파티션으로 나뉘고, 각 파티션이 그 로그 하나다. 오프셋은 파티션 안에서만 증가하므로 **순서 보장도 파티션 안에서만 성립한다**. 실습 토픽 `public.complaint.received.v1`은 파티션이 3개이고 키는 지역구다. 그래서 정확한 문장은 "강남구 민원끼리는 접수 순서가 보존되지만, 강남구와 관악구 사이의 순서는 보장되지 않는다"이다.

실제 실행 기록에서 처음 네 건의 경로를 그대로 옮기면 다음과 같다.

이벤트	키(지역구)	배정 파티션	받은 오프셋
#1 불법주차	중구	1	0
#2 도로파손	강남구	1	1
#3 도로파손	성동구	0	0
#4 가로등고장	중구	1	2

세 가지가 확인된다. 오프셋은 파티션마다 따로 세므로 #3은 전체로는 세 번째지만 파티션 0에서는 오프셋 0이다. 중구 이벤트 두 건은 모두 파티션 1로 가서 그 안에서 순서(0→2)가 보존된다. 그리고 중구와 강남구가 같은 파티션 1을 공유하듯, 키가 다르다고 파티션이 다르리라는 보장은 없다.

여기서 흔한 오해가 하나 깨진다. "토픽에 넣은 순서대로 처리된다"고 기대하고 현황판을 만들면, 파티션이 여럿인 순간 그 기대는 성립하지 않는다. 접수 순서대로 목록을 보여 줘야 한다면 무엇끼리의 순서인지를 먼저 정해야 하고, 그것을 정하는 장치가 다음 절의 파티션 키다.

4. 파티션 키는 순서 보장 단위의 선택이다

키를 붙이면 생산자가 그 값을 해시(hash) 함수로 숫자로 바꾸고, 그 숫자를 파티션 개수로 나눈 나머지로 파티션 번호를 정한다. 같은 키는 늘 같은 나머지가 나오므로 늘 같은 파티션으로 간다. 이 계산은 브로커가 아니라 생산자 쪽에서 일어난다.

그래서 키를 고르는 일은 설정값 고르기가 아니라 **"무엇끼리 순서가 지켜져야 하는가"를 정하는 업무 판단**이다. 지역구를 키로 하면 지역구별 접수 순서가 보존되어 구 단위 집계와 감사에 맞고, 민원 건 ID를 키로 하면 한 민원의 접수→배정→처리 순서가 보존되어 건 단위 이력 추적에 맞는다. 키를 지정하지 않으면 부하는 고르게 퍼지지만 어떤 순서도 보장되지 않는다.

실측을 보면 키의 성질이 그대로 드러난다. 8개 자치구를 키로 200건을 보냈더니 전송에 0.301초가 걸렸고, 파티션 0에 61건, 파티션 1에 76건, 파티션 2에 63건이 들어갔다. 완전히 균등했다면 $200 \div 3 = 66.7$ 건씩이어야 하는데, 파티션 1이 나머지 둘보다 약 20% 많다.

이 불균등의 원인은 지역구별 민원 발생량 차이가 아니라 **키 개수와 파티션 수가 나누어떨어지지 않는다는 것**이다. 같은 산출물의 `district_to_partitions` 를 보면 8개 구가 2:3:3으로 갈렸다 — 관악·성동이 파티션 0, 강남·마포·중이 파티션 1, 송파·용산·종로가 파티션 2다. 구별 이벤트 수가 비슷하면 세 구를 받은 파티션이 두 구를 받은 파티션의 1.5배를 받고, 61:76:63이 그 비율에 가깝다.

`key_to_single_partition: true` 는 각 구의 이벤트가 한 파티션에만 들어갔다는 뜻이다. 이 값이 참이면 불균등의 원인을 발생량 차이로 돌릴 수 없다. 구마다 파티션이 하나로 고정되어 있으므로 남은 원인은 해시 배정뿐이고, 동시에 이 값은 구별 접수 순서가 보존되었다는 증거이기도 하다.

키 선택에는 대가가 따른다. 특정 구에 재난이 나서 그 구의 민원이 수십 배로 늘면, 같은 키는 같은 파티션이라는 성질 때문에 그 이벤트가 전부 한 파티션에 몰린다. 이것이 핫 파티션(hot partition)이고, 나머지 두 파티션에 여유가 있어도 부하를 나눠 가질 방법이 없다. 순서 보장 범위를 좁힐수록(예: 민원 건 ID) 부하는 고르게 퍼지지만 넓은 범위의 순서는 포기하게 된다.

파티션 수를 나중에 늘리는 것도 같은 계산식 때문에 위험하다. 파티션이 3개에서 4개가 되면 나누는 수가 바뀌어 나머지도 바뀌므로, "강남구"가 배정되는 파티션 번호가 달라질 수 있다. 그러면 같은 구의 과거 이벤트와 새 이벤트가 다른 파티션에 놓여 구별 접수 순서가 그 시점에서 끊긴다. 감사에서 "그날의 접수 순서를 재현하라"는 요구가 오면 재현할 수 없으므로, 초기 설계에서 파티션 수에 여유를 둔다.

토픽 설계표에 들어가는 값 — 토픽 이름, 파티션 키, 순서 보장 단위, 파티션 수, 보존 기간(retention) — 은 이렇게 요구사항에서 나온다. 보존 기간은 디스크 사정이 아니라 감사 주기에서 역산한다. 감사가 주 1회 누락을 확인한다면 보존은 최소 그 주기를 덮어야 하고, 실습 토픽의 예시 값은 7일이다.

5. 커밋 시점이 전달 보증을 결정한다

같은 `group.id` 를 가진 소비자들은 하나의 컨슈머 그룹(consumer group)이 되어 파티션을 나눠 맡는다. 그룹이 다르면 같은 토픽을 독립적으로 읽으므로, 처리용 그룹과 감사 적재용 그룹이 서로를 방해하지 않는다. 각 그룹은 "어디까지 읽었다"를 브로커에 커밋(commit)하고, 재시작하면 그 위치부터 다시 읽는다.

그래서 처리와 커밋 중 무엇을 먼저 하느냐가 장애 때의 결과를 가른다. 두 가지 선택뿐이다.

커밋 먼저, 처리 나중(at-most-once, 최대 한 번). 처리 중 크래시(crash)하면 재시작한 소비자는 커밋 이후부터 읽으므로 처리하지 못한 레코드는 다시 오지 않는다. 유실이 가능하고 중복은 없다.

처리 먼저, 커밋 나중(at-least-once, 최소 한 번). 커밋 전에 크래시하면 옛 커밋 위치부터 다시 읽으므로 이미 처리한 레코드가 다시 온다. 유실이 없고 중복이 가능하다.

실습에서 이 두 순서를 각각 강제로 중단시켜 보면 결과가 예측대로 걸린다. 커밋을 먼저 한 시나리오 B는 30건을 확보·커밋한 뒤 10건만 처리하고 종료했고, 재시작한 소비자는 커밋 위치 뒤부터 읽어 아무것도 받지 못했다 — **20건이 유실됐고 오류 로그는 남지 않았다**. 처리를 먼저 한 시나리오 C는 커밋 없이 10건을 처리하고 종료했고, 재시작 후 30건을 처음부터 다시 읽어 **중복 10건**이 기록됐다(처리 기록 40건, 고유 이벤트 30건, 유실 0).

여기서 "정확히 한 번(exactly-once)을 쓰면 되지 않느냐"는 질문이 나온다. 그런데 네트워크 너머의 침묵은 "기록에 실패했다"와 "기록은 됐는데 응답만 유실됐다"를 구분해 주지 않으므로, 물리적으로 정확히 한 번 전달하는 것은 근본적으로 어렵다. Kafka의 exactly-once는 전달이 아니라 **반영**을 뜻한다 — 여러 번 전달되어도 결과가 한 번 반영된 것과 같게 만든다는 의미이며, 소비 종단의 저장소까지 협조해야 완성되고 처리량 비용도 따른다.

한 가지 함정을 덧붙인다. 자동 커밋(enable.auto.commit=true)을 켜 두면 백그라운드가 일정 주기로 커밋하므로, 저장소에 실제로 쓰기 전에 오프셋이 밀려 올라갈 수 있다. at-least-once를 의도한 코드가 조용히 at-most-once로 강등되어 유실이 나는데, 오류 한 줄 없이 진행된다. 그래서 실습은 자동 커밋을 끄고 커밋 시점을 코드로 정한다.

복구에도 시간이 든다. 소비자가 응답 없이 멈추면 브로커는 세션 타임아웃(session timeout, 실습 설정 10초)이 지나야 그 소비자를 그룹에서 빼고 파티션을 다른 구성원에게 재할당한다(리밸런싱, rebalancing). 그래서 크래시 직후 재시작한 소비자는 곧바로 파티션을 받지 못하며, 실습 스크립트도 12초를 기다린 뒤 재시작한다. 장애 복구 시간에는 프로세스 재기동만이 아니라 이 재할당 대기가 포함된다.

6. 유실과 중복은 비대칭이다

두 실패는 대칭이 아니다. 중복은 세기 쉽고 교정할 수 있지만, 유실은 세기 어렵고 되돌릴 수 없다. 중복은 중복 제거 키가 충돌한 건수로 계수하면 되고, 시나리오 C의 중복 10건도 산출물에 그대로 기록됐다.

유실은 그렇지 않다. 시나리오 B의 20건은 어디에도 오류로 남지 않았고, 처리 기록에는 10건이 정상 처리된 것으로만 보인다. 없어진 것을 세려면 없어진 것을 알아야 하는데, 유실의 정의상 그 기록이 없다.

그래서 유실을 잡는 수단은 대사(reconciliation) 하나다. 시간 창을 정해 "이 창에서 생산자가 보낸 건수"와 "소비자가 처리한 고유 건수"를 비교하고, 차이가 0이 아니면 조사한다. 이 대사를 자동화하지 않으면 유실은 감사 시점이나 시민의 항의로 발견된다.

이 비대칭이 공공 데이터의 기본값을 정한다. 민원 처리에서 at-most-once를 쓰면 "접수됐는데 처리 기록이 없는 민원"이 생기고, 이것은 복구할 방법이 없다. at-least-once를 쓰면 같은 민원이 두 번 처리 목록에 오르지만 중복 제거로 교정된다. 그래서 **실무 기본값은 at-least-once로 유실을 막고, 중복은 하류에서**

제거하는 것이며 그 하류 설계가 5장의 주제다.

다만 양쪽 다 감내하지 못하는 업무도 있다. 재난 문자는 누락되면 인명 위험이고 중복 발령되면 경보 자체의 신뢰가 깎인다. 이럴 때는 둘 중 하나를 고르는 대신 발송 게이트웨이에 멍등 키를 부여해 양쪽을 모두 차단한다. 판단의 형식은 같고 답이 도메인마다 다르다.

7. 실습 4.1 민원 이벤트 수집 파이프라인 설계와 실행 (난이도

★★★★☆☆)

실습 목표

토픽 이름·파티션 키·파티션 수·보존 기간을 스스로 정해 근거를 적고, 실제 브로커에서 세 시나리오를 실행한다. 예측한 파티션 분포를 실행 로그와 대조하고, 커밋 시점을 바꾼 두 크래시로 유실과 중복을 직접 발생시켜 관찰한다.

민원 이벤트는 시뮬레이션 입력이다(개인정보가 포함된 실제 민원은 사용할 수 없다). 관찰 대상은 데이터 내용이 아니라 실제 브로커의 전달 동작이다.

1단계 설계 — 실행 전에 적는다

요구사항은 이렇다. 서울시 8개 자치구에서 민원이 접수되고 이벤트에는 `complaint_id`, `district`, `category`, `received_at` 이 담긴다. 구청별 현황판이 접수 순서대로 목록을 보여 준다. 감사는 주 1회 접수 누락 여부를 확인한다. 재난 시 특정 구의 민원이 평시의 수집 배로 늘 수 있다.

다음 여섯 항목을 정하고 각각 요구사항의 어느 문장에서 나왔는지 적는다.

- 토픽 이름
- 파티션 키
- 순서 보장 단위
- 파티션 수
- 보존 기간
- 이 설계가 깨지는 조건

근거 칸에 "기본값이라서"를 쓰지 않는다. 보존 기간은 감사 주기에서 역산한다. 깨지는 조건에는 자기 선택이 불리해지는 상황을 적는다.

이어서 파티션별 예상 건수를 적는다. 실행 스크립트는 파티션 3개짜리 토픽에 지역구 키로 200건을 보낸다. 실행하기 전에 파티션 0.1.2의 예상 건수와 그 근거 한 줄을 써 둔다.

시작 전 준비

이 실습은 **Docker Desktop**이 실행 중이어야 한다. 아래 한 줄이 Docker 상태와 포트를 점검하고, 가상환경과 패키지를 준비한 뒤 Kafka-Zookeeper 컨테이너까지 띄운다.

```
python scripts/setup_practice.py 4
```

브로커는 3장에서 만든 최소 구성을 재사용한다. 컨테이너 이미지를 처음 내려받으면 몇 분 걸리므로 수업 전에 한 번 실행해 두는 편이 낫다. 2181:9092:29092 포트를 다른 프로그램이 쓰고 있으면 스크립트가 알려 준다.

2단계 실행

3장 최소 구성의 브로커를 재사용한다.

```
cd practice/chapter3 && docker compose up -d zookeeper kafka
cd ../chapter4
python3 -m venv venv && source venv/bin/activate
pip install -r code/requirements.txt
python run_chapter4.py
```

커밋 시점 차이의 구현은 다음이 전부다.

```
if mode == "at-most-once":
    consumer.commit()      # 처리 "전" 커밋 → 크래시 시 유실
    process(batch)
if mode == "at-least-once":
    consumer.commit()      # 처리 "후" 커밋 → 크래시 시 중복
```

전체 코드는 `practice/chapter4/code/4-2-kafka-consumer.py`, 오케스트레이션은 `practice/chapter4/run_chapter4.py` 참고

오케스트레이터는 세 시나리오를 차례로 실행한다. A는 소비자를 먼저 띄우고 200건을 생산하는 정상 경로이고, B는 커밋을 먼저 하는 소비자가 10건 처리 후 강제 종료(exit 42)되며, C는 처리를 먼저 하는 소비자가 같은 지점에서 강제 종료된다. 크래시는 `os._exit()` 로 재현하는데, 커밋도 그룹 탈퇴 통지도 없이 멈추는 실제 장애와 같은 조건이다.

3단계 대조 — 예측과 실측 맞춰 보기

관찰 보고서는 `practice/chapter4/data/output/ch4_delivery_report.json` 에 저장된다.

표 4.7 시나리오별 유실·중복·지연

시나리오	생산	처리 기록	고유 이벤트	중복	유실	비고
A. baseline	200	200	200	0	0	지연 평균 3.5ms·최대 110.7ms
B. at-most-once+크래시	30	10	10	0	20	커밋 먼저 → 유실
C. at-least-once+크래시	30	40	30	10	0	처리 먼저 → 재수신 중복

A의 종단 지연 최대 110.7ms는 평균 3.5ms의 약 32배인데, 처리 기록을 보면 이 최댓값은 가장 첫 레코드에서 나왔다. 첫 폴링에는 파티션 할당과 패치 준비 같은 시동 비용이 붙는다. 지연은 평균만으로 판단하지 않으며, 최댓값이 언제 나왔는지가 원인 진단의 단서다.

파티션 분포 예측은 `ch4_produce_baseline.json`의 `partition_distribution` 값으로 채운다. 실측은 61:76:63이고 균등값은 66.7이다. 예측을 균등으로 적었다면 4절을 다시 읽는다. 키 기반 파티셔닝은 이벤트를 고르게 흘리는 장치가 아니라 같은 키를 같은 파티션에 묶는 장치이고, 부하 균등은 키가 충분히 많고 빈도가 비슷할 때 따라오는 부수 효과다.

유실·중복 건수는 폴링 배치 크기와 크래시 시점에 따라 실행마다 달라질 수 있다. 확인할 것은 특정 숫자가 아니라 방향이다. B는 반드시 유실 쪽으로, C는 반드시 중복 쪽으로 실패한다.

마지막으로 1단계의 "이 설계가 깨지는 조건"을 다시 쓴다. 처음에 막연히 적었더라도 지금은 실측 분포를 근거로 쓸 수 있다. 제출하는 설계는 이 수정을 반영한 최종본이다.

산출물

`ch4_delivery_report.json` 이 시나리오별 생산·처리·유실·중복·지연을 담은 관찰 보고서이고, `ch4_produce_*.json` 이 파티션 분포와 키→파티션 매핑의 원본 증거다. `ch4_consume_*.json` 과 `ch4_processed_*.jsonl` 은 소비 결과와 이벤트별 처리 기록이며 5장 실습의 입력이 된다.

8. 핵심 요약

- 1. 순서는 토픽이 아니라 파티션 안에서만 지켜진다. 오프셋이 파티션마다 따로 매겨지기 때문이다. 워크드 예제의 #3은 전체로는 세 번째지만 파티션 0에서는 오프셋 0이며, 중구 두 건은 같은 파티션 1 안에서만 순서가 보존된다.
- 2. 키는 부하 분산 장치가 아니라 순서 보장 단위의 선택이다. 8개 구가 3개 파티션에 2:3:3으로 갈려 61:76:63이 나왔고, 균등값 66.7과 벌어진 원인은 발생량 차이가 아니라 해시 배정이다. `key_to_single_partition: true` 가 그 판별 근거이자 구별 접수 순서가 보존됐다는 증거다.
- 3. 커밋 시점 하나가 유실과 중복을 가르친다. 커밋을 먼저 한 시나리오 B는 20건을 유실했고, 처리를 먼저 한 시나리오 C는 10건을 중복 처리했다. 같은 코드에서 두 줄의 순서만 바꾼 결과이며, 자동 커밋을 켜 두면 이 선택이 조용히 뒤집힌다.

4. **exactly-once**는 "정확히 한 번 전달"이 아니라 "정확히 한 번 반영"이다. 응답이 오지 않은 이유가 실패인지 응답 유실인지 구분할 수 없으므로 물리적 1회 전달은 보장되지 않는다. 따라서 "Kafka를 쓰므로 유실도 중복도 없다"는 발주 문장은 사실과 다르다.

5. 유실과 중복은 비대칭이므로 공공 데이터는 **at-least-once**에서 출발한다. 중복은 세어서 교정할 수 있지만 유실은 오류 로그조차 남기지 않는다. 유실을 잡는 수단은 생산 건수와 처리 고유 건수를 맞춰 보는 대사뿐이고, 남은 중복은 5장의 멍등 설계로 흡수한다.

9. 과제

실습 4.1을 실행해 생성된 `practice/chapter4/data/output/ch4_delivery_report.json` 을 그대로 LMS에 업로드하고, 1단계 설계 여섯 항목(3단계 대조를 반영한 최종본)을 함께 제출한다.

이어서 본인 산출물의 값을 인용해 두 가지를 각각 세 문장 이내로 쓴다.

1. 본인의 `A_baseline.produced` 와 `partitions` 로 균등값을 계산하고, `partition_distribution` 이 그 값에서 파티션별로 얼마나 벗어났는지 적는다. 그 차이의 원인이 지역구별 발생량 차이가 아님을 `key_to_single_partition` 값을 근거로 서술한다.
2. 본인의 `B_at_most_once_crash.lost_events` 와 `C_at_least_once_crash.duplicated_events` 를 인용해, 두 시나리오의 커밋 시점 차이가 어떻게 이 두 값을 만들었는지 설명한다.

수업일 포함 7일 이내에 제출한다.

10. 더 알아보기

이 장에서 다루지 않은 내용은 실무교재 `docs/chapter4.md` 에 있다.

- **acks 설정과 멍등 Producer** — `acks=0/1/all` 의 유실 위험 비교, `enable.idempotence` 가 막는 중복의 범위: `docs/chapter4.md` §4.3
- **단계별 실패 지점 표와 exactly-once의 구현 축** — 트랜잭션으로 읽기·처리·쓰기·커밋을 하나로 묶는 방식: §4.5
- **토픽 명명 규칙과 설계표 예시 3종, 민간 패턴의 공공 적용** — 광고 집계·주문 결제 패턴을 공공 업무로 옮길 때 바뀌는 근거: §4.2, §4.6
- **위험 식별 표를 검수 언어로 옮기는 형식** — 실습 4단계의 전체 서술과 제출물 3종: 실습 4.1 4단계